



API Rate Limiting in .NET



Ck Nitin

Jun 17

8.6k

1

10

100

Article

In the world of web APIs, especially public ones, controlling how many requests a user or client can make is very important. This is known as **API Rate Limiting**. Without it, one careless or malicious user could overload your server, slow down services for others, or even bring down your entire application.

Let's understand this concept deeply, see how it works in .NET, and explore some best scenarios, real examples, and even interview questions that can help you.

What is API Rate Limiting?

Simply put, **Rate Limiting** controls how often someone (like a user, client, or IP address) can make requests to an API within a specific period.

For example

- A public API may allow **100 requests per minute** per user.
- If someone sends 101 requests, the 101st will be rejected or delayed.

This protects the backend and ensures **fair usage** by all clients.

Why Do We Need API Rate Limiting?

1. **Avoid Server Overload:** Prevents your server from crashing due to too many requests.
2. **Stops Abuse:** Blocks bots or malicious users from hammering your API.
3. **Ensures Fair Usage:** All users get a fair share of the service.
4. **Reduces Cost:** If you're using cloud services (like Azure/AWS), more requests mean more cost.
5. **Enhances Security:** Prevents brute-force attacks, especially on login APIs.

API Rate Limiting in .NET

.NET provides simple and flexible ways to implement rate limiting, especially since **.NET 7** and **.NET 8**, where a built-in **Rate Limiting Middleware** was introduced.

Setting Up Rate Limiting in ASP.NET Core 8



1. Install NuGet Package (if required)

```
1 dotnet add package AspNetCoreRateLimit
```

2. Configure in Program.cs

```
1 builder.Services.AddMemoryCache();
2 builder.Services.Configure<IpRateLimitOptions>(builder.Configuration.GetSe
3 builder.Services.AddSingleton<IRateLimitConfiguration, RateLimitConfiguratio
4 builder.Services.AddInMemoryRateLimiting();
```

In appsettings.json

```
1 "IpRateLimiting": {
2   "EnableEndpointRateLimiting": true,
3   "StackBlockedRequests": false,
4   "RealIpHeader": "X-Real-IP",
5   "ClientIdHeader": "X-ClientId",
6   "HttpStatusCode": 429,
7   "GeneralRules": [
8     {
9       "Endpoint": "*",
10      "Period": "1m",
11      "Limit": 5
12    }
13  ]
14 }
```

✓ This will limit each client to **5 requests per minute** for all endpoints (*).

Alternate way

```
1 using Microsoft.AspNetCore.RateLimiting;
2 using System.Threading.RateLimiting;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddRateLimiter(options =>
7 {
8   options.AddFixedWindowLimiter("fixed", opt =>
9     r
```



```
12     opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
13     opt.QueueLimit = 2; // Maximum 2 requests will wait in queue
14 });
15 });
16
17 var app = builder.Build();
18
19 app.UseRateLimiter();
20
21 app.MapGet("/", () => "Hello!").RequireRateLimiting("fixed");
22
23 app.Run();
```

Explanation

- **Window**: Time window to measure requests.
- **PermitLimit**: Number of requests allowed.
- **QueueLimit**: Extra requests to wait in the queue.
- **RequireRateLimiting("fixed")**: Apply this policy to specific endpoints.

Real-world Scenarios (Where Rate Limiting Helps)

1. Public APIs like weather, currency exchange, etc.

- Prevents free users from abusing the API.

2. Login & Authentication endpoints

- Stops brute-force attacks.

3. Payment Gateway APIs

- Limits transaction requests to avoid fraud.

4. Microservices architecture

- One service calling another repeatedly can exhaust system resources.

Example Use Case

Imagine a **Matrimony Application API**:

- Free users: Max 20 requests/minute.
- Premium users: Max 100 requests/minute.

This ensures premium users get faster service without being affected by free users' heavy



How to do this?

Use user roles or JWT claims inside the limiter logic to apply different limits dynamically.

How to Handle Clients Exceeding Limits?

When a client hits the limit:

- **429 Too Many Requests** HTTP status code is returned.
- Optionally, include a `Retry-After` header to inform them when they can try again.

Example response

1	HTTP/1.1 429 Too Many Requests
2	<code>Retry-After: 30</code>

Best Practices for API Rate Limiting

- ☒ Set different limits for different users (e.g., free vs. paid).
- ☒ Always log when limits are hit (for audit or debugging).
- ☒ Notify clients properly with error codes and retry headers.
- ☒ Use Redis or distributed caches for scalable limits across multiple servers.
- ☒ Don't forget internal APIs or microservices calls — they may also need limits.

Common Interview Questions (with Answers)

Q 1. Why is Rate Limiting important in APIs?

A. It prevents misuse, protects backend resources, avoids server overload, and ensures fair usage among clients.

Q 2. How do you implement Rate Limiting in .NET Core?

A. Using the `AspNetCore.RateLimiting` middleware. Configure policies (`FixedWindow`, `SlidingWindow`, `TokenBucket`, `Concurrency`) in the `Program.cs` file and apply them to endpoints using `.RequireRateLimiting("policyName")`.

Q 3. What is the difference between Fixed Window and Sliding Window rate limiting?

A

- **Fixed Window:** Limits apply to fixed time blocks (e.g., 10 requests every minute).



A. 429 Too Many Requests.

Q 5. Can Rate Limiting be role-based?

A. Yes. You can set different limits based on user roles, API keys, or tokens using custom resolvers or policies.

Conclusion

Rate limiting is no longer optional — it's a must-have feature for all APIs. In .NET, adding rate limiting is now super simple thanks to the built-in middleware. Whether you're building small internal tools or large public services, a proper rate-limiting strategy will protect your application from abuse and ensure reliability.

People also reading

-  Understanding API Rate Limiting in the context of web development. 
-  The role of API Rate Limiting in enhancing security and reducing cost. 
-  Exploring API Rate Limiting in .NET. 
-  Applying API Rate Limiting in real-world scenarios. 
-  Best practices and common interview questions regarding API Rate Limiting. 

(1) Comments

[View All Comments](#)



Type your comment here and press Enter Key (Minimum 10 characters)



Jumpstart Blockchain Build with JavaScript

©2025 C# Corner. All contents are copyright of their authors.